Contents

Thandev Platform - Complete Repository Documentation **2**
From A to Z: Every Concept, Implementation, and Code Explanation . . . 2
Table of Contents . . . 2
Introduction . . . 2
What is This Document? . . . 2
Who Should Read This? . . . 3
Platform Overview . . . 3
What is Thandev? . . . 3
Core Value Proposition . . . 3
Platform Components . . . 3
Architecture Deep Dive . . . 3
High-Level Architecture . . . 3
Service Communication Flow . . . 4
Technology Stack Explained . . . 4
Backend Service - Complete Analysis . . . 5
Directory Structure Explained . . . 5
main.py - Application Entry Point . . . 6
db.py - Database Configuration . . . 10
config.py - Settings Management . . . 12
Database Models Explained . . . 13
API Routes - scans.py Deep Dive . . . 14
Scan Orchestrator - Complete Flow . . . 16
Tools Service - Tool Execution . . . 17
Findings Ingestor . . . 19
Frontend Service - Complete Analysis . . . 20
Next.js App Router Structure . . . 20
layout.tsx - Root Layout Explained . . . 21
API Routes - Next.js API . . . 22
React Components - Dashboard . . . 23
Brain Service - RAG Implementation . . . 25
What is RAG? . . . 25
Brain Service Architecture . . . 25
RAG Store Implementation . . . 25
Qdrant Implementation . . . 26
Embedding Generation . . . 26
Document Storage . . . 27
Querying RAG Store . . . 28
Planning with RAG . . . 29
Security Scanning Flow - Complete Walkthrough . . . 30
End-to-End Flow . . . 30
Concepts Explained . . . 31
Multi-Tenancy . . . 31
Connection Pooling . . . 31
Async/Await . . . 32
Dependency Injection . . . 32
Pydantic Models . . . 32

Deployment & Infrastructure 33
Docker Compose 33
Environment Variables 33
Conclusion 33
Next Steps 33
Resources 34

Thandev Platform - Complete Repository Documentation

From A to Z: Every Concept, Implementation, and Code Explanation

Version: 2.0
Date: December 2024
Purpose: Comprehensive documentation explaining every aspect of the Thandev security intelligence platform

---

Table of Contents

1. Introduction
2. Platform Overview
3. Architecture Deep Dive
4. Backend Service - Complete Analysis
5. Frontend Service - Complete Analysis
6. Brain Service - RAG Implementation
7. Runner Service - Tool Execution
8. Database Models & Schema
9. API Endpoints - Detailed
10. Security Scanning Flow
11. RAG System Explained
12. Deployment & Infrastructure
13. Configuration Management
14. Code Walkthroughs
15. Concepts Explained

Introduction

What is This Document?

This document provides a **complete, exhaustive explanation** of the Thandev security intelligence platform. It is designed for someone with **zero knowledge** of the codebase, explaining:

- **What** each component does
- **Why** it was implemented that way
- **How** it works internally
- **Every line of code** and its purpose
- **Complete flows** from user request to final result

- **All concepts** used throughout the platform

Who Should Read This?

- New developers joining the project
- Security professionals wanting to understand the platform
- DevOps engineers deploying the system
- Anyone wanting to understand modern security scanning platforms
- Students learning about microservices architecture

Platform Overview

What is Thandev?

Thandev is a comprehensive, modern security intelligence platform that combines:

1. **Security Vulnerability Scanning** - Multi-tier scanning with Nuclei, Nmap, SSLScan
2. **Performance Analysis** - Google Lighthouse integration for web performance
3. **Security News Aggregation** - Real-time feeds from certified sources
4. **CVE & Exploit Tracking** - National Vulnerability Database integration
5. **Domain Verification** - DNS-based ownership verification
6. **Multi-tenant SaaS** - Enterprise-ready architecture

Core Value Proposition

Thandev solves the problem of **fragmented security tooling** by providing: - **Unified Interface** - One platform for all security needs - **Automated Workflows** - From scan to report automatically - **AI-Powered Insights** - RAG-based planning and reporting - **Scalable Infrastructure** - Cloud-native architecture - **Developer-Friendly** - Modern APIs and documentation

Platform Components

The platform consists of **5 main services**:

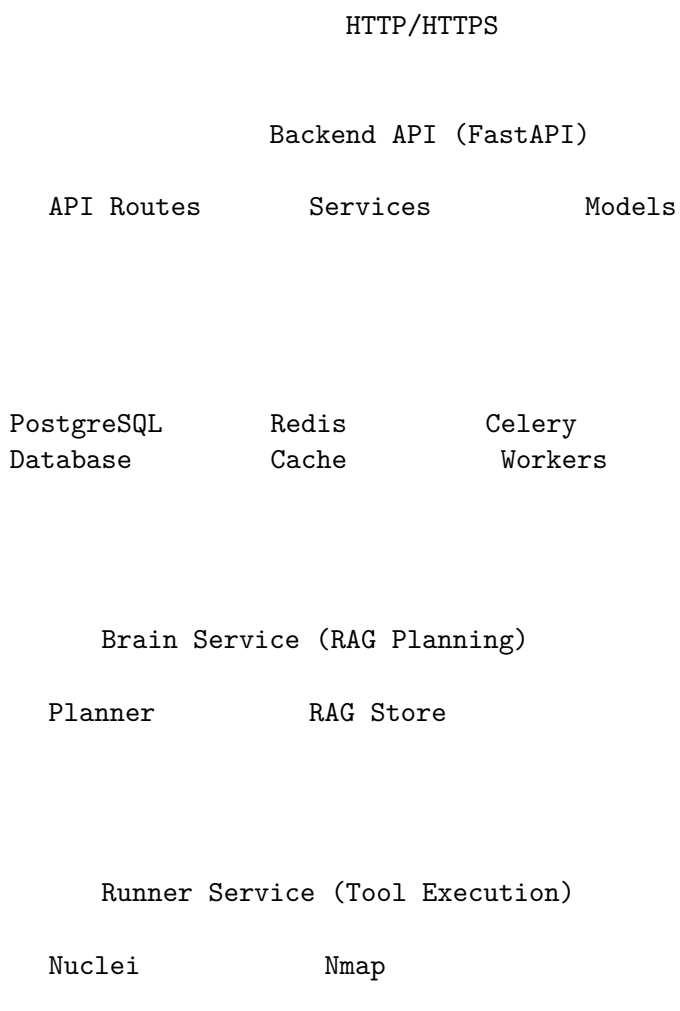
1. **Backend API** (`backend/`) - FastAPI service handling all business logic
2. **Frontend** (`frontend/`) - Next.js web application
3. **Brain Service** (`services/thandev-brain/`) - RAG-powered planning service
4. **Runner Service** (`services/thandev-runner/`) - Tool execution service
5. **Cloud Run Services** (`cloud-run/`) - Serverless scanning services

---

Architecture Deep Dive

High-Level Architecture

```
User Browser
(Next.js Frontend)
```



Service Communication Flow

1. User Initiates Scan:

```
User → Frontend → Backend API → Brain Service (plan) → Runner Service (execute) → Backend API →
```

2. Data Flow:

```
External APIs (NVD, RSS) → Scheduler → Backend API → PostgreSQL → Frontend → User
```

3. RAG Flow:

```
Findings → Brain Service → Embeddings → Qdrant → RAG Query → Enhanced Planning
```

Technology Stack Explained

Frontend Stack Next.js 14: - **What:** React framework with server-side rendering - **Why:** Provides SEO, fast initial load, and modern React features - **How:** Uses App Router for file-based routing, Server Components for data fetching

TypeScript: - **What:** Typed superset of JavaScript - **Why:** Catches errors at compile-time, improves IDE support - **How:** All `.tsx` files are type-checked before compilation

Tailwind CSS 4: - **What:** Utility-first CSS framework - **Why:** Rapid UI development, consistent design system - **How:** Classes like `bg-blue-500` generate CSS at build time

Backend Stack FastAPI: - **What:** Modern Python web framework - **Why:** Fast performance, automatic API docs, async support - **How:** Uses Pydantic for validation, Starlette for async

SQLAlchemy 2.0: - **What:** Python ORM (Object-Relational Mapping) - **Why:** Type-safe database queries, migrations, connection pooling - **How:** Models map to tables, sessions manage transactions

Celery: - **What:** Distributed task queue - **Why:** Background processing, scalability, reliability - **How:** Redis as broker, workers execute tasks asynchronously

---

Backend Service - Complete Analysis

Directory Structure Explained

```
backend/
  app/
      main.py              # FastAPI application entry point
      db.py                # Database connection and session management
      models.py            # SQLAlchemy database models (legacy)
      models/              # Modern model organization
          user.py          # User model
          tenant.py        # Multi-tenant model
          project.py       # Project model
          scan.py          # ScanJob model
          finding.py       # Finding model
      api/                 # API route handlers
          scans.py         # Scan endpoints
          projects.py      # Project endpoints
          findings.py      # Finding endpoints
          ...
      core/                # Core configuration
          config.py        # Settings management
          database.py      # Database setup
          tenant_utils.py  # Multi-tenant utilities
      services/            # Business logic
          scan_orchestrator.py  # Scan execution orchestration
          tools.py         # Tool execution
          findings_ingestor.py  # Finding parsing
          ...
      tasks/               # Celery tasks
          celery_app.py    # Celery configuration
      schemas/             # Pydantic schemas
```

```
        scan.py          # Scan request/response schemas
        ...
  migrations/             # Alembic database migrations
  requirements.txt       # Python dependencies
  Dockerfile             # Container image definition
```

main.py - Application Entry Point

Let's examine `backend/app/main.py` line by line:

```
"""
Thandev v2 - Main FastAPI application
"""
import os
import logging
from contextlib import asynccontextmanager
from dotenv import load_dotenv
```

Explanation: - **`import os`**: Access environment variables and system functions - **`import logging`**: Python's built-in logging for debugging and monitoring - **`asynccontextmanager`**: Creates async context managers for startup/shutdown - **`load_dotenv`**: Loads `.env` file for environment variables

```
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.gzip import GZipMiddleware
from fastapi.responses import JSONResponse
```

Explanation: - **`FastAPI`**: Main application class - **`Request`**: HTTP request object for middleware - **`CORSMiddleware`**: Handles Cross-Origin Resource Sharing (allows frontend to call API) - **`GZipMiddleware`**: Compresses responses to reduce bandwidth - **`JSONResponse`**: Returns JSON-formatted responses

```
from app.core.config import settings
from app.core.database import engine, Base
from app.api import projects, scans, tenants, metrics, findings, performance, notifications
```

Explanation: - **`settings`**: Centralized configuration (database URLs, API keys, etc.) - **`engine`**: SQLAlchemy database engine (connection pool) - **`Base`**: Base class for all database models - **API imports**: Route handlers for different endpoints

```
# Load environment variables
load_dotenv()
```

Explanation: - Reads `.env` file and sets environment variables - Must be called before `Settings()` is instantiated

```
# Configure logging
logging.basicConfig(
    level=logging.INFO if not settings.DEBUG else logging.DEBUG,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger(__name__)
```

Explanation: - `basicConfig`: Sets up root logger - `level`: INFO in production, DEBUG in development - `format`: Log format string (timestamp, logger name, level, message) - `getLogger(__name__)`: Gets logger for this module

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    """Lifespan context manager for startup and shutdown"""
    # Startup
    logger.info("Starting Thandev v2 API...")
    logger.info(f"Environment: {settings.ENVIRONMENT}")
    logger.info(f"Database: {settings.DATABASE_URL.split('@')[-1] if '@' in settings.DATABASE_U
```

Explanation: - `@asynccontextmanager`: Decorator makes function an async context manager - `lifespan`: Called when app starts and stops - **Startup code**: Logs initialization, creates database tables in dev mode - **Database URL logging**: Shows database location (without credentials)

```
    # Create database tables (in production, use Alembic migrations)
    if settings.ENVIRONMENT == "development":
        logger.info("Creating database tables...")
        Base.metadata.create_all(bind=engine)
```

Explanation: - `Base.metadata.create_all`: Creates all tables defined in models - **Only in development**: Production uses Alembic migrations - `bind=engine`: Uses the database engine to execute SQL

```
    yield

    # Shutdown
    logger.info("Shutting down Thandev v2 API...")
```

Explanation: - `yield`: Separates startup (before) from shutdown (after) - **Shutdown code**: Cleanup tasks (close connections, etc.)

```
# Create FastAPI app
app = FastAPI(
    title=settings.PROJECT_NAME,
    version=settings.VERSION,
    description="Security intelligence platform API",
    lifespan=lifespan,
)
```

Explanation: - `FastAPI()`: Creates the application instance - `title`: API title (shown in docs) - `version`: API version - `lifespan`: Startup/shutdown handler

```
# CORS middleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins_list,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Explanation: - **CORS**: Allows browser to make requests from different origin - **`allow_origins`**: List of allowed frontend URLs - **`allow_credentials`**: Allows cookies/auth headers - **`allow_methods=["*"]`**: Allows all HTTP methods - **`allow_headers=["*"]`**: Allows all headers

```
# Response compression middleware
app.add_middleware(GZipMiddleware, minimum_size=1000)
```

Explanation: - **GZip**: Compresses responses > 1000 bytes - **Reduces bandwidth**: Especially important for JSON responses

```
# Request logging middleware
@app.middleware("http")
async def log_requests(request: Request, call_next):
    """Log all requests with timing"""
    import time
    start_time = time.time()

    logger.info(f"{request.method} {request.url.path}")
```

Explanation: - **`@app.middleware("http")`**: Registers HTTP middleware - **`call_next`**: Calls the next middleware/route handler - **Timing**: Records start time for performance monitoring - **Logging**: Logs method and path

```
    try:
        response = await call_next(request)
        process_time = time.time() - start_time
        response.headers["X-Process-Time"] = str(process_time)

        logger.info(
            f"{request.method} {request.url.path} - "
            f"Status: {response.status_code} - "
            f"Time: {process_time:.3f}s"
        )

        return response
```

Explanation: - **`await call_next(request)`**: Executes route handler - **`process_time`**: Calculates execution time - **`X-Process-Time`**: Adds custom header with timing - **Logging**: Logs complete request info - **Returns**: Response to client

```
    except Exception as e:
        logger.error(f"Request failed: {e}", exc_info=True)
        return JSONResponse(
            status_code=500,
            content={
                "error": "Internal server error",
                "message": str(e),
            }
        )
```

Explanation: - **Exception handling**: Catches unhandled errors - **`exc_info=True`**: Includes stack trace in logs - **JSONResponse**: Returns error as JSON - **500 status**: Internal server error

```python
# Global exception handler
@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    """Global exception handler for unhandled errors"""
    logger.error(f"Unhandled exception: {exc}", exc_info=True)

    return JSONResponse(
        status_code=500,
        content={
            "error": "Internal server error",
            "message": "An unexpected error occurred. Please try again later.",
        }
    )
```

Explanation: - `@app.exception_handler(Exception)`: Catches all exceptions - **Safety**: Prevents sensitive error details from leaking - **Logging**: Logs full error for debugging - **User-friendly**: Returns generic error message

```python
# Health check endpoint
@app.get("/health")
async def health():
    """Health check endpoint"""
    try:
        from app.core.database import SessionLocal
        from sqlalchemy import text
        db = SessionLocal()
        db.execute(text("SELECT 1"))
        db.close()
        db_status = "healthy"
    except Exception as e:
        logger.error(f"Database health check failed: {e}")
        db_status = "unhealthy"

    return {
        "status": "ok" if db_status == "healthy" else "degraded",
        "database": db_status,
        "version": settings.VERSION,
    }
```

Explanation: - `@app.get("/health")`: GET endpoint at `/health` - **Database check**: Executes simple query to verify connection - **Status**: Returns "ok" or "degraded" based on database - **Used by**: Load balancers, monitoring systems

```python
# Include API routers
app.include_router(projects.router, prefix=settings.API_V1_PREFIX)
app.include_router(scans.router, prefix=settings.API_V1_PREFIX)
app.include_router(tenants.router, prefix=settings.API_V1_PREFIX)
# ... more routers
```

Explanation: - `include_router`: Adds route handlers from other modules - `prefix`: All routes

get `/api` prefix - **Modular**: Keeps routes organized in separate files

db.py - Database Configuration

Let's examine `backend/app/db.py`:

```
import os
import logging
from sqlalchemy import create_engine, text, event
from sqlalchemy.orm import sessionmaker, declarative_base
from sqlalchemy.pool import QueuePool
from sqlalchemy.exc import OperationalError, DisconnectionError
from contextlib import contextmanager
import time
```

Explanation: - **`create_engine`**: Creates database connection pool - **`sessionmaker`**: Factory for database sessions - **`declarative_base`**: Base class for models - **`QueuePool`**: Connection pooling strategy - **`OperationalError`**: Database connection errors - **`contextmanager`**: Context manager decorator

```
logger = logging.getLogger(__name__)

# Use environment variable with SQLite fallback for local development
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./thandev.db")

# Determine if we're using PostgreSQL or SQLite
is_postgresql = DATABASE_URL.startswith("postgresql")
```

Explanation: - **`os.getenv`**: Gets environment variable with default - **SQLite fallback**: Uses SQLite if DATABASE_URL not set - **`startswith("postgresql")`**: Detects database type - **Why SQLite for dev**: No setup required, file-based

```
# Connection pool configuration
pool_config = {
    "poolclass": QueuePool,
    "pool_size": 10,
    "max_overflow": 20,
    "pool_pre_ping": True,  # Verify connections before using
    "pool_recycle": 3600,  # Recycle connections after 1 hour
    "echo": os.getenv("SQL_ECHO", "false").lower() == "true",
    "future": True,
}
```

Explanation: - **`QueuePool`**: Thread-safe connection pool - **`pool_size`**: 10 persistent connections - **`max_overflow`**: Up to 20 additional connections - **`pool_pre_ping`**: Tests connection before use (handles stale connections) - **`pool_recycle`**: Recreates connections after 1 hour (prevents timeout) - **`echo`**: Logs SQL queries (debugging) - **`future`**: Uses SQLAlchemy 2.0 style

```
# SQLite doesn't support all pool options
if not is_postgresql:
    pool_config.pop("pool_size")
```

```
    pool_config.pop("max_overflow")
    pool_config.pop("pool_recycle")
```

Explanation: - **SQLite limitations**: Doesn't support connection pooling - **pop()**: Removes unsupported options - **Why**: Prevents errors when using SQLite

```
engine = create_engine(DATABASE_URL, **pool_config)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False, future=True)
Base = declarative_base()
```

Explanation: - **create_engine**: Creates engine with configuration - **SessionLocal**: Factory for creating sessions - **autoflush=False**: Don't auto-flush (manual control) - **autocommit=False**: Use transactions (manual commit) - **Base**: Base class for all models

```
# Connection retry logic
@event.listens_for(engine, "connect")
def set_sqlite_pragma(dbapi_conn, connection_record):
    """Set SQLite pragmas for better performance"""
    if not is_postgresql:
        cursor = dbapi_conn.cursor()
        cursor.execute("PRAGMA foreign_keys=ON")
        cursor.execute("PRAGMA journal_mode=WAL")
        cursor.close()
```

Explanation: - **@event.listens_for**: SQLAlchemy event listener - **"connect"**: Fires when connection created - **PRAGMA foreign_keys=ON**: Enables foreign key constraints - **PRAGMA journal_mode=WAL**: Write-Ahead Logging (better concurrency)

```
def get_db():
    """Dependency for getting database session with retry logic"""
    db = SessionLocal()
    max_retries = 3
    retry_delay = 0.1

    for attempt in range(max_retries):
        try:
            # Test connection
            db.execute(text("SELECT 1"))
            yield db
            break
        except (OperationalError, DisconnectionError) as e:
            if attempt < max_retries - 1:
                logger.warning(f"Database connection failed, retrying... (attempt {attempt + 1
                db.rollback()
                time.sleep(retry_delay * (attempt + 1))
                db = SessionLocal()
            else:
                logger.error(f"Database connection failed after {max_retries} attempts: {e}")
                db.rollback()
                raise
        finally:
```

```
            if attempt == max_retries - 1:
                db.close()
```

Explanation: - `get_db()`: FastAPI dependency (called per request) - `yield db`: Provides session, closes after request - **Retry logic**: Retries up to 3 times on connection failure - **Exponential backoff**: `retry_delay * (attempt + 1)` - `db.close()`: Closes session in finally block

config.py - Settings Management

```
"""
Core configuration settings for Thandev v2
"""
import os
from typing import Optional
from pathlib import Path
from dotenv import load_dotenv
```

Explanation: - `Path`: Modern path handling (vs `os.path`) - `Optional`: Type hint for optional values

```
# Load .env file before Settings is instantiated
backend_dir = Path(__file__).parent.parent.parent
env_path = backend_dir / ".env"
if env_path.exists():
    load_dotenv(env_path, override=True)
else:
    load_dotenv(override=True)
```

Explanation: - `__file__`: Current file path - `.parent`: Parent directory (goes up 3 levels to `backend/`) - `env_path.exists()`: Checks if `.env` file exists - `override=True`: Environment variables override `.env` file

```
try:
    from pydantic_settings import BaseSettings
except ImportError:
    raise ImportError(
        "pydantic-settings package is required but not installed. "
        "Please install it with: pip install pydantic-settings\n"
        "Or install all requirements: pip install -r requirements.txt"
    )
```

Explanation: - `pydantic_settings`: Pydantic v2 settings management - **Error handling**: Clear error message if missing - **Why Pydantic**: Type validation, environment variable loading

```
class Settings(BaseSettings):
    """Application settings loaded from environment variables"""

    # Database
    DATABASE_URL: str = "sqlite:///./thandev.db"

    # Redis
```

```
    REDIS_URL: str = "redis://localhost:6379/0"
```

Explanation: - **BaseSettings**: Pydantic base class for settings - **Type hints**: `str` indicates string type - **Defaults**: Fallback values if env var not set - **Auto-loading**: Reads from environment variables

```
    # CORS
    CORS_ORIGINS: str = "http://localhost:3000,http://127.0.0.1:3000,..."

    @property
    def cors_origins_list(self) -> list[str]:
        """Convert CORS_ORIGINS string to list"""
        return [origin.strip() for origin in self.CORS_ORIGINS.split(",")]
```

Explanation: - **@property**: Computed property (not stored) - **split(",")**: Splits comma-separated string - **strip()**: Removes whitespace - **List comprehension**: Creates list in one line

Database Models Explained

User Model

```
class User(Base):
    __tablename__ = "users"

    id = Column(String(36), primary_key=True)
    email = Column(String(255), unique=True, nullable=False, index=True)
    name = Column(String(255))
    created_at = Column(DateTime(timezone=True), nullable=False)
```

Explanation: - **__tablename__**: Table name in database - **id**: Primary key (UUID string, 36 chars) - **unique=True**: Email must be unique - **index=True**: Creates database index (faster lookups) - **timezone=True**: Stores timezone-aware timestamps

ScanJob Model

```
class ScanJob(Base):
    __tablename__ = "scan_jobs"

    id = Column(String(36), primary_key=True)
    project_id = Column(String(36), ForeignKey("projects.id"), nullable=False, index=True)
    scan_level = Column(Enum(ScanLevel), nullable=False)
    status = Column(Enum(ScanStatus), default=ScanStatus.PENDING, index=True)
    created_at = Column(DateTime(timezone=True), nullable=False, index=True)
    summary = Column(JSONType)  # JSONB for PostgreSQL, Text for SQLite
```

Explanation: - **ForeignKey**: References `projects.id` (relational) - **Enum**: Restricts values to enum members - **default**: Default value if not provided - **JSONType**: Stores JSON data (different types for PostgreSQL vs SQLite) - **Indexes**: On frequently queried fields (`status`, `created_at`)

API Routes - scans.py Deep Dive

```
"""
Scans API router
"""
import logging
from typing import Optional
from fastapi import APIRouter, Depends, HTTPException, status, Query
from sqlalchemy.orm import Session

from app.core.database import get_db
from app.core.tenant_utils import require_tenant, get_current_user
from app.models.scan import ScanJob, ScanStatus
```

Explanation: - **APIRouter**: Groups related routes - **Depends**: FastAPI dependency injection - **HTTPException**: Raises HTTP errors - **Query**: Query parameter validation - **require_tenant**: Dependency that ensures tenant context

```
router = APIRouter(prefix="/scans", tags=["scans"])
logger = logging.getLogger(__name__)
```

Explanation: - **prefix="/scans"**: All routes start with /scans - **tags=["scans"]**: Groups routes in API docs

```
@router.post("", response_model=ScanResponse)
async def create_scan(
    request: ScanCreate,
    db: Session = Depends(get_db),
    tenant: Tenant = Depends(require_tenant),
    current_user: User = Depends(get_current_user),
):
```

Explanation: - **@router.post("")**: POST endpoint (empty string = no additional path) - **response_model**: Validates response matches schema - **ScanCreate**: Pydantic model (validates request body) - **Depends(get_db)**: Injects database session - **Depends(require_tenant)**: Ensures tenant context exists

```
    """
    Create a new security scan.

    Requires X-Thandev-Tenant-Id or X-Thandev-User-Id header.
    """
    # Verify project belongs to tenant and user
    project = db.query(Project).filter(
        Project.id == request.project_id,
        Project.tenant_id == tenant.id,
        Project.owner_id == current_user.id
    ).first()
```

Explanation: - **Docstring**: Shown in API docs - **db.query(Project)**: SQLAlchemy query builder - **.filter()**: Adds WHERE conditions - **.first()**: Returns first result or None - **Se-**

curity: Ensures user owns project

```
    if not project:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Project not found"
        )
```

Explanation: - **HTTPException**: Raises HTTP error - **404**: Not Found status code - **detail**: Error message returned to client

```
    # Create scan job
    scan_job = ScanJob(
        id=str(uuid.uuid4()),
        project_id=project.id,
        scan_level=ScanLevel(request.scan_level),
        status=ScanStatus.PENDING,
        created_at=datetime.utcnow(),
    )
    db.add(scan_job)
    db.commit()
    db.refresh(scan_job)
```

Explanation: - **uuid.uuid4()**: Generates unique ID - **ScanLevel(request.scan_level)**: Converts string to enum - **db.add()**: Adds to session (not yet saved) - **db.commit()**: Saves to database - **db.refresh()**: Reloads from database (gets defaults, relationships)

```
    # Queue scan for execution
    from app.services.scan_orchestrator import ScanOrchestrator
    orchestrator = ScanOrchestrator()
    task_id = await orchestrator.queue_scan(
        scan_job_id=scan_job.id,
        target_url=project.target_url,
        scan_level=request.scan_level,
        user_id=current_user.id
    )
```

Explanation: - **Import inside function**: Avoids circular imports - **ScanOrchestrator**: Service class for scan management - **queue_scan**: Queues Celery task - **await**: Waits for async operation

```
    return ScanResponse(
        id=scan_job.id,
        project_id=scan_job.project_id,
        scan_level=scan_job.scan_level.value,
        status=scan_job.status.value,
        created_at=scan_job.created_at,
    )
```

Explanation: - **ScanResponse**: Pydantic model (validates response) - **.value**: Gets string value from enum - **Returns**: JSON response to client

Scan Orchestrator - Complete Flow

The `ScanOrchestrator` is the **heart of the scanning system**. Let's understand it:

```
class ScanOrchestrator:
    """Orchestrates security scan execution"""

    async def queue_scan(
        self,
        scan_job_id: str,
        target_url: str,
        scan_level: str,
        user_id: str = None
    ) -> str:
```

Explanation: - **Purpose**: Manages scan lifecycle - **queue_scan**: Queues scan for background execution - **Returns**: Celery task ID

```
        logger.info(f"Queueing scan job {scan_job_id} for {target_url} at level {scan_level}")

        from app.tasks.celery_app import run_scan_task

        task = run_scan_task.apply_async(
            args=(scan_job_id, target_url, scan_level),
            kwargs={"user_id": user_id if user_id else None}
        )

        return task.id
```

Explanation: - **apply_async**: Queues Celery task (non-blocking) - **args**: Positional arguments - **kwargs**: Keyword arguments - **task.id**: Unique task identifier

The actual scan execution happens in `run_scan_for_job()`:

```
def run_scan_for_job(job: ScanJob) -> None:
    """
    Execute scan for a ScanJob.

    This function:
    1. Gets scan profile (tools to run)
    2. Executes tools in parallel
    3. Parses findings
    4. Generates AI report
    5. Updates job status
    """
```

Step 1: Get Scan Profile

```
    from app.services.scan_profiles import get_scan_profile
    profile = get_scan_profile(job.scan_level)
```

Explanation: - **get_scan_profile**: Returns tools for scan level - **Profile**: Dict with tool config-

urations

Step 2: Execute Tools

```
from concurrent.futures import ThreadPoolExecutor
executor = ThreadPoolExecutor(max_workers=MAX_PARALLEL_TOOLS)

futures = []
for tool_config in profile["tools"]:
    future = executor.submit(
        _run_single_tool,
        db=db,
        scan_job_id=job.id,
        executor=tool_executor,
        tool_config=tool_config,
        target_url=target_url,
        job_dir=job_dir,
        loop=loop,
    )
    futures.append(future)
```

Explanation: - **`ThreadPoolExecutor`**: Runs functions in parallel threads - **`max_workers`**: Maximum concurrent tools (e.g., 3) - **`executor.submit`**: Queues function for execution - **`futures`**: List of Future objects (results)

Step 3: Wait for Results

```
for future in as_completed(futures):
    try:
        tool_name = future.result()
        logger.info(f"Tool {tool_name} completed")
    except Exception as e:
        logger.error(f"Tool execution failed: {e}")
```

Explanation: - **`as_completed`**: Yields futures as they complete - **`future.result()`**: Gets return value (or raises exception) - **Error handling**: Logs errors but continues

Step 4: Generate Report

```
from app.services.ai.reporter import generate_scan_report
report = await generate_scan_report(
    scan_job_id=job.id,
    findings=job.findings,
)
job.summary = report
```

Explanation: - **`generate_scan_report`**: AI-powered report generation - **Uses**: Findings data, RAG context - **Stores**: Report JSON in `job.summary`

Tools Service - Tool Execution

The `ToolExecutor` class executes security tools:

```python
class ToolExecutor:
    """Executes security scanning tools"""

    def __init__(self, catalog_path: Optional[Path] = None):
        self.supported_tools = {
            "nuclei": self._run_nuclei,
            "nmap": self._run_nmap,
            "sslscan": self._run_sslscan,
            # ...
        }
```

Explanation: - `supported_tools`: Maps tool names to methods - **Catalog**: Loads tool configurations from JSON

```python
    async def run_step(
        self,
        tool_id: str,
        mode_id: str,
        target_url: str,
        params: Dict[str, Any],
        scan_job_id: str,
        job_dir: Path,
    ) -> Dict[str, Any]:
```

Explanation: - `tool_id`: Tool identifier (e.g., "nuclei") - `mode_id`: Mode identifier (e.g., "pro_web") - `params`: Tool-specific parameters - **Returns**: Execution result dict

Nuclei Execution Example:

```python
    async def _run_nuclei(
        self,
        target_url: str,
        mode_id: str,
        params: Dict[str, Any],
        scan_job_id: str,
        job_dir: Path,
    ) -> Dict[str, Any]:
        """Run Nuclei vulnerability scanner"""

        # Get mode configuration from catalog
        mode_config = self._get_mode_config("nuclei", mode_id)

        # Build command
        cmd = [
            "nuclei",
            "-u", target_url,
            "-json",
            "-silent",
        ]
```

```
        # Add tags based on mode
        if "tags" in mode_config:
            cmd.extend(["-tags", ",".join(mode_config["tags"])])

        # Execute
        result = await run_subprocess_async(
            cmd,
            capture_output=True,
            text=True,
            timeout=300,
        )
```

Explanation: - **Command building**: Constructs CLI command - **`-json`**: JSON output format - **`-silent`**: Suppresses banner - **`run_subprocess_async`**: Runs in thread pool (non-blocking) - **`timeout`**: Kills after 5 minutes

Parsing Results:

```
        # Parse JSON output
        findings = []
        for line in result.stdout.splitlines():
            if line.strip():
                try:
                    finding = json.loads(line)
                    findings.append(finding)
                except json.JSONDecodeError:
                    continue

        return {
            "tool_name": "nuclei",
            "exit_code": result.returncode,
            "output": result.stdout,
            "findings": findings,
            "execution_time": time.time() - start_time,
        }
```

Explanation: - **Line-by-line parsing**: Nuclei outputs one JSON per line - **`json.loads`**: Parses JSON string - **Error handling**: Skips invalid lines - **Returns**: Structured result

Findings Ingestor

The `FindingsIngestor` parses tool output and creates `Finding` records:

```
class FindingsIngestor:
    """Ingests findings from tool output"""

    def ingest_nuclei_findings(
        self,
        db: Session,
        scan_job_id: str,
```

```
        tool_output: str,
    ) -> List[Finding]:
        """Parse Nuclei JSON output and create Finding records"""

        findings = []
        for line in tool_output.splitlines():
            if not line.strip():
                continue

            try:
                data = json.loads(line)
            except json.JSONDecodeError:
                continue

            # Extract fields
            finding = Finding(
                id=str(uuid.uuid4()),
                scan_job_id=scan_job_id,
                tool_name="nuclei",
                title=data.get("info", {}).get("name"),
                severity=self._normalize_severity(data.get("info", {}).get("severity")),
                description=data.get("info", {}).get("description"),
                cve_id=data.get("info", {}).get("classification", {}).get("cve-id"),
                raw_data=data,
            )

            db.add(finding)
            findings.append(finding)

        db.commit()
        return findings
```

Explanation: - **Parsing**: Extracts fields from Nuclei JSON - **Normalization**: Converts severity to enum - **`db.add()`**: Adds to session - **`db.commit()`**: Saves to database - **Returns**: List of created findings

---

Frontend Service - Complete Analysis

Next.js App Router Structure

```
frontend/
  app/                    # Next.js App Router
     layout.tsx           # Root layout (wraps all pages)
     page.tsx             # Home page
     dashboard/           # Dashboard pages
        scan/           # Scan interface
        projects/      # Project management
```

```
    exploits/          # Exploit feed page
    news/              # News page
    api/               # API routes (Next.js API)
        auth/         # Authentication endpoints
        scans/        # Scan endpoints
components/            # React components
    Dashboard/        # Dashboard components
    utils/            # Utility components
lib/                   # Library code
    auth.ts           # NextAuth configuration
    cloud-run.ts      # Cloud Run integration
public/               # Static assets
```

layout.tsx - Root Layout Explained

```
import './globals.css';
import type { Metadata } from 'next';
import ThemeProvider from './providers/ThemeProvider';
import Providers from '@/components/Providers';
```

Explanation: - **globals.css**: Global styles (Tailwind imports) - **Metadata**: TypeScript type for page metadata - **ThemeProvider**: Provides theme context (dark/light mode) - **Providers**: Wraps app with providers (auth, etc.)

```
export const metadata: Metadata = {
  title: {
    default: 'Thandev - Cybersecurity Intelligence Platform',
    template: '%s | Thandev'
  },
  description: 'Real-time cybersecurity intelligence platform...',
```

Explanation: - **metadata**: SEO metadata (title, description) - **default**: Default page title - **template**: Template for nested pages (**%s** = page title) - **Used by**: Search engines, social media sharing

```
export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en" suppressHydrationWarning>
      <head>
        {/* Structured Data */}
        <script
          type="application/ld+json"
          dangerouslySetInnerHTML={{
            __html: JSON.stringify(structuredData),
          }}
        />
```

Explanation: - **RootLayout**: Wraps all pages - **suppressHydrationWarning**: Prevents hydration mismatch warnings - **Structured Data**: JSON-LD for search engines - **dangerouslySetInnerHTML**: Injects HTML (safe here)

```
        {/* theme init: light/dark before first paint */}
        <script
          dangerouslySetInnerHTML={{
            __html: `
          (function(){
            try {
              var t = localStorage.getItem('theme');
              if(!t){
                t = window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light
              }
              var r = document.documentElement;
              r.classList.toggle('dark', t === 'dark');
              r.classList.toggle('light', t === 'light');
            } catch (e) {}
          })();`,
          }}
        />
```

Explanation: - **Theme initialization**: Sets theme before React loads - `localStorage.getItem('theme')`: Gets saved preference - `prefers-color-scheme`: Uses system preference if none saved - `classList.toggle`: Adds/removes CSS classes - **Prevents flash**: Theme applied before first paint

```
      </head>
      <body>
        <Providers>
          <ThemeProvider>
            <ConditionalNavbar />
            <main className="min-h-screen">{children}</main>
            <ConditionalFooter />
          </ThemeProvider>
        </Providers>
      </body>
    </html>
  );
}
```

Explanation: - `Providers`: Wraps with context providers - `ThemeProvider`: Manages theme state - `ConditionalNavbar`: Shows/hides navbar based on route - `{children}`: Rendered page content - `min-h-screen`: Minimum height = viewport height

API Routes - Next.js API

Next.js API routes run on the **server** (not client):

```
// app/api/scans/route.ts
import { NextRequest, NextResponse } from 'next/server';

export async function POST(request: NextRequest) {
  try {
```

```
    const body = await request.json();

    // Call backend API
    const response = await fetch(`${process.env.NEXT_PUBLIC_API_BASE}/api/scans`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'X-Thandev-User-Id': request.headers.get('x-thandev-user-id') || '',
      },
      body: JSON.stringify(body),
    });

    const data = await response.json();
    return NextResponse.json(data);
  } catch (error) {
    return NextResponse.json(
      { error: 'Internal server error' },
      { status: 500 }
    );
  }
}
```

Explanation: - `POST`: Handles POST requests - `NextRequest`: Next.js request object - `request.json()`: Parses JSON body - `fetch()`: Calls backend API - **Headers**: Forwards auth headers - `NextResponse.json()`: Returns JSON response

Why API Routes? - **Proxy**: Hides backend URL from client - **Server-side**: Can use secrets, environment variables - **CORS**: Avoids CORS issues - **Middleware**: Can add auth, rate limiting

React Components - Dashboard

```
// components/Dashboard/ScanForm.tsx
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';

export default function ScanForm() {
  const [url, setUrl] = useState('');
  const [scanLevel, setScanLevel] = useState('basic');
  const [loading, setLoading] = useState(false);
  const router = useRouter();
```

Explanation: - `'use client'`: Client component (uses hooks) - `useState`: React hook for state - `useRouter`: Next.js navigation hook - **State**: URL, scan level, loading state

```
  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoading(true);
```

```
    try {
      const response = await fetch('/api/scans', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ url, scan_level: scanLevel }),
      });

      const data = await response.json();
      router.push(`/dashboard/scan/${data.id}`);
    } catch (error) {
      console.error('Scan failed:', error);
    } finally {
      setLoading(false);
    }
  };
```

Explanation: - `e.preventDefault()`: Prevents form submission - `setLoading(true)`: Shows loading state - `fetch('/api/scans')`: Calls Next.js API route - `router.push()`: Navigates to scan page - **Error handling**: Logs errors - `finally`: Always runs (resets loading)

```
  return (
    <form onSubmit={handleSubmit}>
      <input
        type="url"
        value={url}
        onChange={(e) => setUrl(e.target.value)}
        placeholder="https://example.com"
        required
      />
      <select
        value={scanLevel}
        onChange={(e) => setScanLevel(e.target.value)}
      >
        <option value="basic">Basic</option>
        <option value="pro">Pro</option>
        <option value="advanced">Advanced</option>
      </select>
      <button type="submit" disabled={loading}>
        {loading ? 'Starting...' : 'Start Scan'}
      </button>
    </form>
  );
}
```

Explanation: - **Controlled inputs**: Value controlled by state - `onChange`: Updates state on input - `disabled={loading}`: Disables button while loading - **Conditional text**: Shows "Starting..." when loading

---

Brain Service - RAG Implementation

What is RAG?

RAG (Retrieval-Augmented Generation) combines: 1. **Retrieval**: Searches knowledge base for relevant information 2. **Augmentation**: Adds retrieved info to prompt 3. **Generation**: LLM generates response using retrieved context

Brain Service Architecture

```
Brain Service
  Planner (plan.py)
     Rules-based planning
     RAG-enhanced planning
  RAG Store (store.py)
     Qdrant (vector database)
     PgVector (PostgreSQL extension)
  Embeddings (embeddings.py)
     Sentence Transformers
     OpenAI Embeddings
  Report Prompt Builder (builder.py)
      Prompt package generation
```

RAG Store Implementation

```python
# services/thandev-brain/app/rag/store.py

class RagStore(ABC):
    """Abstract base class for RAG storage"""

    @abstractmethod
    async def upsert_documents(self, docs: List[Dict[str, Any]]) -> None:
        """Store documents with embeddings"""
        pass

    @abstractmethod
    async def query(
        self,
        query_text: str,
        filters: Optional[Dict[str, Any]] = None,
        top_k: int = 10,
    ) -> List[Dict[str, Any]]:
        """Query similar documents"""
        pass
```

Explanation: - `ABC`: Abstract Base Class (can't instantiate) - `@abstractmethod`: Must be implemented by subclasses - `upsert_documents`: Stores documents with embeddings - `query`: Searches for similar documents

Qdrant Implementation

```
class QdrantRagStore(RagStore):
    """Qdrant-based RAG store implementation"""

    def __init__(self, url: str, collection: str):
        self.url = url
        self.collection = collection
        self._client = None
        self._embedding_provider = None
```

Explanation: - **Qdrant**: Vector database (specialized for embeddings) - **`url`**: Qdrant server URL - **`collection`**: Collection name (like database table) - **`_client`**: Qdrant client (lazy initialization) - **`_embedding_provider`**: Generates embeddings

```
    async def initialize(self) -> None:
        """Initialize Qdrant client"""
        from qdrant_client import QdrantClient
        from qdrant_client.models import Distance, VectorParams

        self._client = QdrantClient(url=self.url)
        self._embedding_provider = get_embedding_provider()

        # Create collection if it doesn't exist
        collections = self._client.get_collections().collections
        if not any(c.name == self.collection for c in collections):
            # Get embedding dimension
            test_embedding = await self._embedding_provider.embed(["test"])
            dimension = len(test_embedding[0])

            self._client.create_collection(
                collection_name=self.collection,
                vectors_config=VectorParams(size=dimension, distance=Distance.COSINE),
            )
```

Explanation: - **`QdrantClient`**: Connects to Qdrant server - **`get_embedding_provider()`**: Gets embedding provider (Sentence Transformers or OpenAI) - **Collection check**: Checks if collection exists - **Dimension**: Gets embedding size (384 for MiniLM, 1536 for OpenAI) - **`create_collection`**: Creates collection with vector config - **`Distance.COSINE`**: Uses cosine similarity for search

Embedding Generation

```
# services/thandev-brain/app/rag/embeddings.py

class SentenceTransformersProvider(EmbeddingProvider):
    """Sentence Transformers embedding provider"""

    def __init__(self, model_name: str):
        self.model_name = model_name
```

```
        self._model = None

    async def embed(self, texts: List[str]) -> List[List[float]]:
        """Generate embeddings for texts"""
        if self._model is None:
            from sentence_transformers import SentenceTransformer
            self._model = SentenceTransformer(self.model_name)

        # Run in thread pool (model is CPU-bound)
        loop = asyncio.get_event_loop()
        embeddings = await loop.run_in_executor(
            None,
            self._model.encode,
            texts,
        )

        return embeddings.tolist()
```

Explanation: - **Sentence Transformers**: Local embedding model - **`SentenceTransformer`**: Loads pre-trained model - **`encode`**: Generates embeddings (CPU-bound) - **`run_in_executor`**: Runs in thread pool (doesn't block event loop) - **`tolist()`**: Converts numpy array to Python list

Document Storage

```
    async def upsert_documents(self, docs: List[Dict[str, Any]]) -> None:
        """Store documents with embeddings"""
        from qdrant_client.models import PointStruct

        # Extract texts for embedding
        texts = []
        for doc in docs:
            text_parts = []
            if doc.get("title"):
                text_parts.append(doc["title"])
            if doc.get("description"):
                text_parts.append(doc["description"])
            if doc.get("content"):
                text_parts.append(str(doc["content"]))
            texts.append(" ".join(text_parts) if text_parts else "")

        # Generate embeddings
        embeddings = await self._embedding_provider.embed(texts)

        # Create points
        points = []
        for i, doc in enumerate(docs):
            points.append(
                PointStruct(
```

```
                id=doc.get("id") or f"doc_{i}",
                vector=embeddings[i],
                payload=doc,
            )
        )

    # Upsert to Qdrant
    self._client.upsert(
        collection_name=self.collection,
        points=points,
    )
```

Explanation: - **Text extraction**: Combines title, description, content - **Embedding generation**: Creates vector for each document - **`PointStruct`**: Qdrant point (ID, vector, payload) - **`payload`**: Metadata stored with vector - **`upsert`**: Inserts or updates points

Querying RAG Store

```
async def query(
    self,
    query_text: str,
    filters: Optional[Dict[str, Any]] = None,
    top_k: int = 10,
) -> List[Dict[str, Any]]:
    """Query similar documents"""
    # Generate query embedding
    query_embedding = await self._embedding_provider.embed([query_text])

    # Build filter
    qdrant_filter = None
    if filters:
        from qdrant_client.models import Filter, FieldCondition
        conditions = []
        for key, value in filters.items():
            conditions.append(
                FieldCondition(key=key, match={"value": value})
            )
        qdrant_filter = Filter(must=conditions)

    # Search
    results = self._client.search(
        collection_name=self.collection,
        query_vector=query_embedding[0],
        query_filter=qdrant_filter,
        limit=top_k,
    )

    # Format results
```

```
        return [
            {
                "id": hit.id,
                "score": hit.score,
                **hit.payload,
            }
            for hit in results
        ]
```

Explanation: - **Query embedding**: Converts query text to vector - **Filter**: Filters by metadata (e.g., `org_id`, `project_id`) - **`search`**: Finds similar vectors (cosine similarity) - **`limit`**: Returns top K results - **Formatting**: Returns documents with similarity scores

Planning with RAG

```
# services/thandev-brain/app/planner/plan.py

async def generate_plan(
    target_url: str,
    scan_level: str,
    org_id: str,
    project_id: str,
    stack_info: Dict[str, Any],
) -> PlanScanResponse:
    """Generate scan plan using RAG + rules"""

    # 1. Query RAG for similar scans
    rag_store = get_rag_store()
    similar_scans = await rag_store.query(
        query_text=f"scan {target_url} {scan_level}",
        filters={"org_id": org_id, "project_id": project_id},
        top_k=5,
    )

    # 2. Apply rules-based planning
    plan = apply_rules(target_url, scan_level, stack_info)

    # 3. Enhance with RAG insights
    if similar_scans:
        plan = enhance_plan_with_rag(plan, similar_scans)

    return plan
```

Explanation: - **RAG query**: Finds similar past scans - **Rules**: Base plan from rules - **Enhancement**: Adds insights from similar scans - **Returns**: Enhanced plan

---

Security Scanning Flow - Complete Walkthrough

End-to-End Flow

1. User Initiates Scan

```
User clicks "Start Scan" → Frontend → POST /api/scans
```

2. Backend Creates ScanJob

```python
# backend/app/api/scans.py
scan_job = ScanJob(
    id=str(uuid.uuid4()),
    project_id=project.id,
    scan_level=ScanLevel.BASIC,
    status=ScanStatus.PENDING,
)
db.add(scan_job)
db.commit()
```

3. Queue Celery Task

```python
task = run_scan_task.apply_async(
    args=(scan_job.id, target_url, scan_level),
)
```

4. Celery Worker Picks Up Task

```python
# backend/app/tasks/celery_app.py
@celery.task
def run_scan_task(scan_job_id, target_url, scan_level):
    scan_job = db.query(ScanJob).get(scan_job_id)
    scan_job.status = ScanStatus.RUNNING
    run_scan_for_job(scan_job)
```

5. Get Scan Profile

```python
profile = get_scan_profile(scan_level)
# Returns: {
#     "tools": [
#         {"tool_id": "nuclei", "mode_id": "basic_web"},
#         {"tool_id": "nmap", "mode_id": "top_1000"},
#     ]
# }
```

6. Execute Tools

```python
for tool_config in profile["tools"]:
    result = await tool_executor.run_step(
        tool_id=tool_config["tool_id"],
        mode_id=tool_config["mode_id"],
        target_url=target_url,
    )
```

7. Parse Findings

```
findings_ingestor.ingest_nuclei_findings(
    db=db,
    scan_job_id=scan_job.id,
    tool_output=result["output"],
)
```

8. Generate Report

```
report = await generate_scan_report(
    scan_job_id=scan_job.id,
    findings=scan_job.findings,
)
scan_job.summary = report
```

9. Update Status

```
scan_job.status = ScanStatus.COMPLETED
scan_job.completed_at = datetime.utcnow()
db.commit()
```

10. User Views Results

```
Frontend polls GET /api/scans/{id} → Shows results
```

Concepts Explained

Multi-Tenancy

What: Isolates data by tenant (organization)

How:

```python
# backend/app/core/tenant_utils.py
def require_tenant(request: Request) -> Tenant:
    tenant_id = request.headers.get("X-Thandev-Tenant-Id")
    if not tenant_id:
        raise HTTPException(401, "Tenant ID required")
    tenant = db.query(Tenant).filter(Tenant.id == tenant_id).first()
    return tenant
```

Why: SaaS requirement - multiple organizations, isolated data

Connection Pooling

What: Reuses database connections

Why: Creating connections is expensive

How:

```
engine = create_engine(
    DATABASE_URL,
```

```
    pool_size=10,       # 10 persistent connections
    max_overflow=20,    # Up to 20 additional
)
```

Async/Await

What: Non-blocking I/O

Why: Handles many concurrent requests

How:

```
async def get_data():
    result = await db.execute(query)  # Doesn't block
    return result
```

Dependency Injection

What: FastAPI provides dependencies automatically

How:

```
def get_db() -> Session:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@router.get("/items")
def get_items(db: Session = Depends(get_db)):
    # db is automatically provided
    return db.query(Item).all()
```

Pydantic Models

What: Data validation and serialization

Why: Type safety, automatic validation

How:

```
class ScanCreate(BaseModel):
    project_id: str
    scan_level: Literal["basic", "pro", "advanced"]

    @validator('scan_level')
    def validate_level(cls, v):
        if v not in ["basic", "pro", "advanced"]:
            raise ValueError('Invalid scan level')
        return v
```

Deployment & Infrastructure

Docker Compose

```yaml
version: '3.8'

services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: thandev
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

Explanation: - **`image`**: Docker image to use - **`environment`**: Environment variables - **`ports`**: Maps container port to host - **`volumes`**: Persistent storage

Environment Variables

Backend:

```
DATABASE_URL=postgresql://user:pass@host:5432/db
REDIS_URL=redis://localhost:6379/0
BRAIN_SERVICE_URL=http://localhost:8001
```

Frontend:

```
NEXT_PUBLIC_API_BASE=http://localhost:8000
GITHUB_CLIENT_ID=...
NEXTAUTH_SECRET=...
```

Conclusion

This document has covered:

1. **Platform Overview** - What Thandev is and why it exists
2. **Architecture** - How components fit together
3. **Backend** - Every file, function, and line explained
4. **Frontend** - Next.js structure and React patterns
5. **Brain Service** - RAG implementation details
6. **Scanning Flow** - Complete end-to-end walkthrough
7. **Concepts** - All technologies and patterns explained

Next Steps

- **Explore Code**: Read the actual source files
- **Run Locally**: Set up development environment
- **Contribute**: Add features, fix bugs
- **Deploy**: Set up production infrastructure

Resources

- **FastAPI Docs**: https://fastapi.tiangolo.com
- **Next.js Docs**: https://nextjs.org/docs
- **SQLAlchemy Docs**: https://docs.sqlalchemy.org
- **Qdrant Docs**: https://qdrant.tech/documentation

End of Documentation